Week 6 - Baseline Triage Model (Final)

Vishwesh Pattanaik | CariSurg MedTech Pathways 2026

Two simple baselines (logistic regression + decision tree), evaluated with metrics a clinician can read, compared to a random guess, and then stress-tested with four extra analyses beyond the standard evaluation. Target = `esi` (1 = most urgent, 5 = least). **Random seed = 42.**

1. Data and features

```
from google.colab import drive
drive.mount('/content/drive')
import pandas as pd, numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.dummy import DummyClassifier
from sklearn.metrics import (classification_report, confusion_matrix, ConfusionMatrixDisplay,
                             accuracy_score, f1_score, recall_score)

df = pd.read_csv("/content/drive/MyDrive/yaleemmlc_admissionprediction_triage.csv")
cc = [c for c in df.columns if c.startswith("cc_")]
y = df["esi"].astype(int)
num = ["age","triage_vital_hr","triage_vital_sbp","triage_vital_dbp","triage_vital_rr",
       "triage_vital_o2","triage_vital_temp","triage_glucose","triage_vital_o2_device"]

# 'disposition' is dropped: it is the OUTCOME, known only after triage, so it would leak the answer.
X = pd.concat([df[num], df[cc], pd.get_dummies(df[["gender","arrivalmode"]], drop_first=True)], axis=1)
print("features:", X.shape)
```

```
Mounted at /content/drive
features: (55121, 216)
```

2. Split and scale (80/20 stratified, seed 42)

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
sc = StandardScaler().fit(Xtr[num]); Xtr, Xte = Xtr.copy(), Xte.copy()
Xtr[num] = sc.transform(Xtr[num]); Xte[num] = sc.transform(Xte[num])
```

3. Model 1 - Logistic regression

```
lr = LogisticRegression(max_iter=2000, random_state=42).fit(Xtr, ytr)
pl = lr.predict(Xte)
print("Accuracy:", round(accuracy_score(yte,pl),3),
      " macro F1:", round(f1_score(yte,pl,average='macro'),3),
      " weighted F1:", round(f1_score(yte,pl,average='weighted'),3))
print(classification_report(yte, pl, digits=3))
```

```
Accuracy: 0.685  macro F1: 0.518  weighted F1: 0.679
              precision    recall  f1-score   support

           1      0.800     0.250     0.381        16
           2      0.733     0.618     0.671      3585
           3      0.676     0.777     0.723      5402
           4      0.640     0.622     0.631      1779
           5      0.483     0.115     0.186       243

    accuracy                          0.685     11025
   macro avg      0.666     0.476     0.518     11025
weighted avg      0.685     0.685     0.679     11025
```

4. Model 2 - Decision tree (max_depth = 6)

Depth is bounded on purpose. With 216 features an unbounded tree memorises the training set. Depth 6 keeps the tree interpretable and prevents that. I tested depths 4 to 12: deeper trees raised aggregate scores only by fitting the common classes better, not by catching the urgent ones.

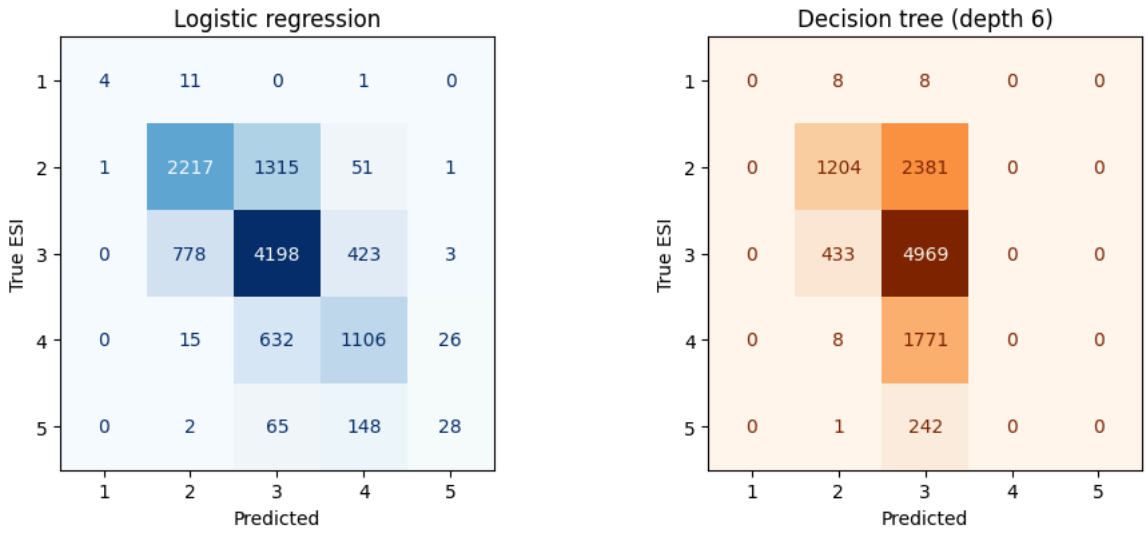
```
dt = DecisionTreeClassifier(max_depth=6, random_state=42).fit(Xtr, ytr)
pt = dt.predict(Xte)
print("Accuracy:", round(accuracy_score(yte,pt),3),
      " macro F1:", round(f1_score(yte,pt,average='macro'),3))
print(classification_report(yte, pt, digits=3))
```

```
Accuracy: 0.56  macro F1: 0.226
              precision    recall  f1-score   support

           1      0.000     0.000     0.000        16
           2      0.728     0.336     0.460      3585
           3      0.530     0.920     0.673      5402
           4      0.000     0.000     0.000      1779
           5      0.000     0.000     0.000       243

    accuracy                          0.560     11025
   macro avg      0.252     0.251     0.226     11025
weighted avg      0.497     0.560     0.479     11025

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

The tree is worse than logistic regression and, worryingly, predicts ESI 3 for almost everyone. It never flags an ESI 1. Logistic regression is the better baseline.

5. Confusion matrices

```
fig, ax = plt.subplots(1,2,figsize=(10,4.2))
ConfusionMatrixDisplay.from_predictions(yte,pl,labels=[1,2,3,4,5],cmap="Blues",ax=ax[0],colorbar=False,values_format="d")
ax[0].set_title("Logistic regression"); ax[0].set_xlabel("Predicted"); ax[0].set_ylabel("True ESI")
ConfusionMatrixDisplay.from_predictions(yte,pt,labels=[1,2,3,4,5],cmap="Oranges",ax=ax[1],colorbar=False,values_format="d")
ax[1].set_title("Decision tree (depth 6)"); ax[1].set_xlabel("Predicted"); ax[1].set_ylabel("True ESI")
plt.tight_layout(); plt.show()
```



6. Random-guess baseline (beat the coin flip)

```
dummy = DummyClassifier(strategy="stratified", random_state=42).fit(Xtr, ytr)
pd_ = dummy.predict(Xte)
print("Random guess :", round(accuracy_score(yte,pd_),3), " macro F1:", round(f1_score(yte,pd_,average='macro'),3))
print("Logistic reg :", round(accuracy_score(yte,pl),3), " macro F1:", round(f1_score(yte,pl,average='macro'),3))
```

```
Random guess : 0.375  macro F1: 0.204
Logistic reg : 0.685  macro F1: 0.518
```

7. Macro vs weighted F1

- **Weighted F1 (0.68)** averages each class by how many patients it has, so the big ESI 2/3 groups dominate.

- **Macro F1 (0.52)** averages the classes equally, so the rare urgent classes count just as much. The gap between them is the warning: the model looks fine on the common patients and poor on the rare, dangerous ones. For triage safety, macro F1 and per-class recall are the honest numbers.

8. Extra analysis 1 - Under-triage vs over-triage (the clinical error that matters)

Not all mistakes are equal. Predicting a patient **less** urgent than they are (under-triage) can be fatal. Predicting them **more** urgent (over-triage) wastes resources but is recoverable.

```
def decompose(p, name):
    over  = (p < yte).mean()
    exact = (p == yte).mean()
    under = (p > yte).mean()
    crit  = yte <= 2
    dangerous = ((p >= 3) & crit).sum() / crit.sum()   # true ESI 1-2 sent to 3+
    print(f"{name:22s} over={over:.3f}  correct={exact:.3f}  under={under:.3f}  "
          f"dangerous under-triage (ESI1-2 -> 3+)={dangerous:.3f}")
decompose(pl,"Logistic regression"); decompose(pt,"Decision tree")
```

```
Logistic regression    over=0.149  correct=0.685  under=0.166  dangerous under-triage (ESI1-2 -> 3+)=0.380
Decision tree          over=0.223  correct=0.560  under=0.217  dangerous under-triage (ESI1-2 -> 3+)=0.663
```

Logistic regression sends 38% of the sickest (ESI 1-2) patients to a non-urgent band. The tree sends 66%. This is the number a clinician actually cares about.

9. Extra analysis 2 - The recall lever (class weighting)

`class_weight='balanced'` tells the model to care about rare classes. It is the single most useful lever for a triage model.

```
lrb = LogisticRegression(max_iter=2000, random_state=42, class_weight="balanced").fit(Xtr, ytr)
plb = lrb.predict(Xte)
print("plain    - ESI1 recall:", round(recall_score(yte,pl,labels=[1],average=None)[0],3),
      " accuracy:", round(accuracy_score(yte,pl),3))
print("weighted - ESI1 recall:", round(recall_score(yte,plb,labels=[1],average=None)[0],3),
      " accuracy:", round(accuracy_score(yte,plb),3))
decompose(plb,"LogReg (weighted)")
```

```
plain    - ESI1 recall: 0.25  accuracy: 0.685
weighted - ESI1 recall: 0.688  accuracy: 0.585
LogReg (weighted)      over=0.173  correct=0.585  under=0.242  dangerous under-triage (ESI1-2 -> 3+)=0.279
```

Weighting roughly triples ESI-1 recall (0.25 to 0.69) and nearly halves dangerous under-triage, at the cost of overall accuracy (0.685 to 0.585). That trade, catching more critical patients in exchange for more false alarms, is exactly the decision the ED Board must make, not the model.

10. Extra analysis 3 - How reliable is the ESI-1 recall? (bootstrap)

There are only ~16 ESI-1 patients in the test set, so a single recall number is shaky. A bootstrap re-samples the test set many times to put a confidence interval around it.

```
rng = np.random.default_rng(42)
yv, pv = yte.values, pl
idx = np.arange(len(yv)); recs = []
for _ in range(2000):
    s = rng.choice(idx, len(idx), replace=True)
    m = yv[s] == 1
    if m.sum() > 0: recs.append((pv[s][m] == 1).mean())
recs = np.array(recs)
print(f"ESI-1 recall = {recall_score(yte,pl,labels=[1],average=None)[0]:.3f}"
      f"   95% CI [{np.percentile(recs,2.5):.3f}, {np.percentile(recs,97.5):.3f}]"
      f"   (only {(yte==1).sum()} ESI-1 patients in test)")
```

```
ESI-1 recall = 0.250   95% CI [0.059, 0.500]   (only 16 ESI-1 patients in test)
```

The interval is very wide (about 0.06 to 0.50). We genuinely cannot pin down ESI-1 recall with so few cases. That argues for more data and for caution before trusting any single figure on the rarest class.

11. Extra analysis 4 - Does the model agree with physiology?

A model can be accurate for the wrong reasons. Here I fit a simple urgent (ESI 1-2) vs not model and check whether each feature pushes in the direction a clinician would expect.

```
yb = (y <= 2).astype(int)
Xtr2,_,ytr2,_ = train_test_split(X, yb, test_size=0.2, stratify=yb, random_state=42)
Xtr2 = Xtr2.copy(); Xtr2[num] = sc.transform(Xtr2[num])
lb = LogisticRegression(max_iter=2000, random_state=42).fit(Xtr2, ytr2)
coef = pd.Series(lb.coef_[0], index=X.columns)
print("(+ pushes toward URGENT)")
for f in ["cc_alteredmentalstatus","cc_chestpain","cc_shortnessofbreath","age","triage_vital_hr",
          "triage_vital_rr","triage_vital_o2","cc_sorethroat","cc_dentalpain"]:
    print(f"  {f:26s} {coef[f]:+.3f}")
```

```
(+ pushes toward URGENT)
  cc_alteredmentalstatus     +2.544
  cc_chestpain               +1.582
  cc_shortnessofbreath       +1.134
  age                        +0.406
  triage_vital_hr            +0.236
  triage_vital_rr            +0.102
  triage_vital_o2            -0.128
  cc_sorethroat              -1.742
  cc_dentalpain              -2.255
```

Every feature points the right way: altered mental status, chest pain, shortness of breath, higher heart and respiratory rate, older age and lower oxygen all raise urgency; sore throat and dental pain lower it. The model learned real clinical patterns, not noise.

12. Verdict, primary metric and failure mode

Primary metric: recall on the urgent classes, above all ESI 1 (equivalently, keeping dangerous under-triage low). Overall accuracy is misleading here because it is dominated by the common ESI 3 group and ignores the 16 sickest patients.

Failure mode I am most worried about: under-triage of ESI 1. The plain baseline misses three quarters of Level 1 patients. In a real ED that is a critically ill patient, an early heart attack or sepsis, sent to the waiting room instead of resuscitation. Class weighting reduces the miss rate but does not remove it, and with only 16 test cases we cannot yet measure it precisely. The baseline clears Dr De Freitas' bar (it beats a coin flip) but is nowhere near safe to deploy, which is the honest message for the Board.